WEEK-1

HTML Introduction

✅ Program 1

Title:
“Design a Restaurant Menu Web Page Using HTML Lists (`ul` and `ol`)”

Concept Covered: Lists, Basic HTML Tags

✅ Program 2

Title:
“Create a Student Registration Form Using HTML Form Elements”

Concept Covered: Forms (`form`, `input`, `select`, `radio`, `submit`)

✅ Program 3

Title: Create Marks sheet table using <table> tag

Concept covered: <table>, <tr>, <th>, <<td>

WEEK-2

✅ Program 4

Title:
“Display a Welcome Message Using HTML DOM and Button Click Event”

Concept Covered: HTML DOM, JavaScript Interaction

✅ Program 5

Title:
“Design a Simple HTML Login Page with JavaScript Validation”

Concept Covered: HTML DOM, JavaScript Interaction

✅ Program 6: Display Images with Captions

Title: “Create a Web Page Displaying Images and Captions”
Concept Covered: Images (`<img>`), Captions (`<figcaption>`), `<div>`/`<span>`

WEEK-3

✅ Program 7: Display Chessboard

Title: Program to display chess board pattern using HTML and CSS

Concepts Covered: HTML Tags – , <style>, CSS Selctors – ID, Class

WEEK-4

Experiment 1: JavaScript Variables and Data Types

Aim: **JavaScript provides different keywords for declaring variables. This experiment demonstrates scope differences between var, let, and const and shows various data types such as Number, String, Boolean, Object, and Array.**

Procedure

1. **Declare variables using var, let, and const.**
2. **Assign different data types.**
3. **Display values using console.log() and alert().**
4. **Observe scope behavior inside and outside blocks.**

Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Experiment 1</title>
</head>
<body>
<script>
  var a = 10;
  let b = "JavaScript";
  const c = true;
```

```
    let obj = { name: "Alice", age: 20 };
    let arr = [1, 2, 3, 4];

    console.log("Number:", a);
    console.log("String:", b);
    console.log("Boolean:", c);
    console.log("Object:", obj);
    console.log("Array:", arr);

    alert("Check console for data types");

    if (true) {
        var x = "Var Scope";
        let y = "Let Scope";
        const z = "Const Scope";
        console.log(x, y, z);
    }
    console.log(x); // Accessible
    // console.log(y); // Error
</script>
</body>
</html>
```

Result

Different data types and scope behavior of var, let, and const were successfully displayed in the console.

Create the usage of var, let, and const and to display different JavaScript data types.

Experiment 2: Control Statements and Functions

Aim To check whether a given number is prime using JavaScript functions.

Experiment 3: Arrays and Objects

Aim To store and manipulate student details using arrays and objects.

Experiment 4: ES6 Features

Aim To demonstrate ES6 features in JavaScript. 1. Use arrow functions.

2. Use function expressions and declaration.

Week-5

Experiment 1 : Callback, Promises, Asynchronous java script

🔹 Aim

To write a JavaScript program demonstrating the use of a **callback function**.

🔹 Description:

A **callback function** is a function that is passed as an argument to another function and is executed after the completion of that function.

🔹 Procedure

1. Define a function greet() that accepts a name and a callback function.
2. Inside greet(), print a greeting message.
3. Call the callback function after printing the greeting.
4. Define another function sayBye() to print a goodbye message.
5. Pass sayBye as a callback when calling greet().

Code:

```
function greet(name, callback) {
   console.log("Hi " + name);
   callback();
}

function sayBye() {
   console.log("Goodbye!");
}

greet("Aman", sayBye);
```

Expected Output:

Hi Aman
Goodbye!

🔹 Aim

To write a JavaScript program demonstrating the use of **Promises**.

🔹 Description

A **Promise** is an object in JavaScript that represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

A Promise has three states:

- Pending
- Fulfilled (Resolved)
- Rejected

🔹 Procedure

1. Create a Promise using the `Promise` constructor.
2. Inside the Promise, define a condition.
3. If the condition is true, call `resolve()`.
4. If false, call `reject()`.
5. Use `.then()` to handle success.
6. Use `.catch()` to handle errors.

Code:

```
let myPromise = new Promise(function(resolve, reject) {
    let success = true;

    if (success) {
        resolve("Task completed!");
    } else {
        reject("Task failed!");
    }
});

myPromise
    .then(function(result) {
        console.log(result);
```

```
})
.catch(function(error) {
    console.log(error);
});
```

Expected Output:

Task completed!

(If `success = false`, output will be `Task failed!`)

🔹 **Aim**

To write a JavaScript program demonstrating **asynchronous execution using async/await**.

🔹 **Description**

Asynchronous JavaScript allows code to run without blocking the execution of other code. It is mainly used for tasks like API calls, file reading, and timers.

✅ What is async/await?

- async is a keyword used to declare an asynchronous function.
- await pauses the execution of the async function until the Promise is resolved.

🔹 **Procedure**

Create a function fetchData() that returns a Promise.

Use setTimeout() to simulate delay.

Use resolve() to return data after delay.

Create an async function.

Use await to wait for the Promise result.

Print the result.

Code:

```
function fetchData() {
    return new Promise(function(resolve) {
        setTimeout(function() {
```

```
            resolve("Data loaded!");
        }, 2000);
    });
}

async function displayData() {
    console.log("Loading...");
    let result = await fetchData();
    console.log(result);
}

displayData();
```

Expected output:

Loading...
(After 2 seconds)
Data loaded!

Experiment 2:Design a calculator application using JavaScript.

Aim:

To design and develop a simple calculator application using **HTML, CSS, and JavaScript** that performs basic arithmetic operations like addition, subtraction, multiplication, and division.

Description:

The Calculator Application is a web-based program developed using:

- **HTML** → To create the structure of the calculator (buttons and display).
- **CSS** → To design and style the calculator interface.
- **JavaScript** → To implement the logic for performing calculations.

The application allows users to:

- Enter numbers using buttons
- Perform arithmetic operations (+, –, ×, ÷)
- View results instantly
- Clear the display

The calculator uses JavaScript functions to capture button clicks and evaluate mathematical expressions dynamically.

Procedure

Step 1: Create HTML Structure

- Create a main container for the calculator.
- Add an <input> field to display numbers and results.
- Add buttons for:
    - Numbers (0–9)
    - Operators (+, –, *, /)

- Equal (=)
- Clear (C)

Step 2: Apply CSS Styling

- Center the calculator using Flexbox.
- Add padding, border-radius, and box-shadow.
- Style buttons with equal width and height.
- Align buttons properly in rows.

Step 3: Add JavaScript Functions

1. appendValue(value)

- Adds clicked numbers/operators to the display.

2. clearDisplay()

- Clears the display when "C" button is clicked.

3. calculate()

- Evaluates the mathematical expression.
- Displays result using eval() function.
- Uses try-catch to handle errors.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple Calculator</title>
    <style>
        body {
            font-family: Arial;
            display: flex;
```

```
            justify-content: center;
            align-items: center;
            height: 100vh;
            background: #f4f4f4;
        }

        .calculator {
            background: white;
            padding: 40px;
            border-radius: 10px;
            box-shadow: 0 0 10px gray;
        }

        #display {
            width: 100%;
            height: 40px;
            margin-bottom: 10px;
            font-size: 18px;
            text-align: right;
            padding-right: 10px;
        }

        button {
            width: 50px;
            height: 40px;
            margin: 5px;
            font-size: 18px;
            cursor: pointer;
        }

        .row {
            text-align: center;
        }
    </style>
</head>
<body>

<div class="calculator">
    <input type="text" id="display" disabled>
```

```
    <div class="row">
        <button onclick="clearDisplay()">C</button>
        <button onclick="appendValue('/')">/</button>
        <button onclick="appendValue('*')">*</button>
        <button onclick="appendValue('-')">-</button>
    </div>

    <div class="row">
        <button onclick="appendValue('7')">7</button>
        <button onclick="appendValue('8')">8</button>
        <button onclick="appendValue('9')">9</button>
        <button onclick="appendValue('+')">+</button>
    </div>

    <div class="row">
        <button onclick="appendValue('4')">4</button>
        <button onclick="appendValue('5')">5</button>
        <button onclick="appendValue('6')">6</button>
        <button onclick="calculate()">=</button>
    </div>

    <div class="row">
        <button onclick="appendValue('1')">1</button>
        <button onclick="appendValue('2')">2</button>
        <button onclick="appendValue('3')">3</button>
        <button onclick="appendValue('0')">0</button>
    </div>
</div>

<script>
    function appendValue(value) {
        document.getElementById("display").value += value;
    }

    function clearDisplay() {
        document.getElementById("display").value = "";
    }

    function calculate() {
        let display = document.getElementById("display");
```

```
        try {
            display.value = eval(display.value);
        } catch {
            display.value = "Error";
        }
    }
</script>

</body>
</html>
```

Week-6

Create a responsive web page of your class time table by using the bootstrap grid system.

Aim of the Program:

The aim of this program is to create a **class timetable** using **HTML** and **Bootstrap**. The timetable displays days and time slots with color-coded cells for easy readability.

Description of the Program:

This program uses HTML and Bootstrap to display a **responsive lab timetable** with days of the week and time periods. The timetable cells are color-coded to distinguish between class periods and breaks (such as lunch).

Code

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Lab Timetable</title>

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.
css" rel="stylesheet">

    <style>
```

```
        body { background: #f8f9fa; }

        .header-box { border: 2px solid #000; padding: 15px; text-align:
center; }

        table { border: 2px solid #000; }

        table th, table td { border: 1px solid #000; text-align: center;
height: 60px; }

        .break { background-color: #f8c1a4; writing-mode: vertical-rl;
transform: rotate(180deg); font-weight: bold; }

        .class-period { background-color: #d1e7fd; }

        .alternate-period { background-color: #e2f9e1; }

    </style>

</head>

<body>



<div class="container mt-4">

    <!-- Header Section -->

    <div class="header-box">

        <h3>ABC Engineering College</h3>

        <h5>Department of Computer Science</h5>

        <p><strong>Effective Date:</strong> 01-03-2026</p>

    </div>



    <!-- Timetable -->

    <div class="table-responsive">
```

```
<table class="table">
    <thead class="table-light">
        <tr>
            <th>Day</th>
            <th>9:00 - 10:00</th>
            <th>10:00 - 11:00</th>
            <th>11:00 - 12:00</th>
            <th>12:00 - 1:00</th>
            <th>1:00 - 2:00</th>
            <th>2:00 - 3:00</th>
            <th>3:00 - 4:00</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <th>Monday</th>
            <td colspan="2" class="class-period"></td>
            <td class="alternate-period"></td>
            <td class="break" rowspan="5">Lunch Break</td>
            <td class="class-period"></td>
            <td class="alternate-period"></td>
            <td class="class-period"></td>
        </tr>
```

```
            <tr>
                <th>Tuesday</th>
                <td class="class-period"></td>
                <td class="alternate-period"></td>
                <td class="class-period"></td>
                <td class="class-period"></td>
                <td class="alternate-period"></td>
                <td class="class-period"></td>
            </tr>
            <tr>
                <th>Wednesday</th>
                <td class="class-period"></td>
                <td class="alternate-period"></td>
                <td class="class-period"></td>
                <td class="alternate-period"></td>
                <td class="class-period"></td>
                <td class="alternate-period"></td>
            </tr>
            <tr>
                <th>Thursday</th>
                <td class="class-period"></td>
                <td class="alternate-period"></td>
                <td class="class-period"></td>
```

```
                    <td class="class-period"></td>
                    <td class="alternate-period"></td>
                    <td class="class-period"></td>
                </tr>
                <tr>
                    <th>Friday</th>
                    <td class="class-period"></td>
                    <td class="alternate-period"></td>
                    <td class="class-period"></td>
                    <td class="alternate-period"></td>
                    <td class="class-period"></td>
                    <td class="alternate-period"></td>
                </tr>
            </tbody>
        </table>
    </div>
</div>


</body>
</html>
```

Create a timer component to start, pause and reset using ReactJS.

```
import React, { useState } from "react";

function Counterapp() {
  const [count, setCount] = useState(5);

  return (
    <button onClick={() => setCount(count + 5)}>
      count is {count}
    </button>
  );
}

export default Counterapp
```

Week-7

Program-1 Create a timer component start, pause and reset using ReactJS.

```
import React, { useState, useEffect } from "react";

function Counterapp() {
  const [count, setCount] = useState(0);
  const [running, setRunning] = useState(false);

  useEffect(() => {
    if (!running) return;

    const interval = setInterval(() => {
      setCount((c) => c + 1);
    }, 1000);

    return () => clearInterval(interval);
  }, [running]);

  return (
    <div style={styles.container}>
      <div style={styles.box}>
        <h2>Count: {count}</h2>

        <button onClick={() => setRunning(true)}>Start</button>
        <button onClick={() => setRunning(false)}>Pause</button>
        <button onClick={() => {
          setRunning(false);
          setCount(0);
        }}>
          Reset
        </button>
      </div>
    </div>
  );
}

const styles = {
```

```
  container: {
    display: "flex",
    justifyContent: "center",  // horizontal center
    alignItems: "center",      // vertical center
    height: "100vh",           // full screen height
  },
  box: {
    textAlign: "center",
  },
};

export default Counterapp;
```

Output:



Program-2 Demonstrate the state props example using React.

```
import React, { useState } from "react";

// Car component (receives props)
function Car(props) {
  return <h3>I am a {props.brand} car</h3>;
}

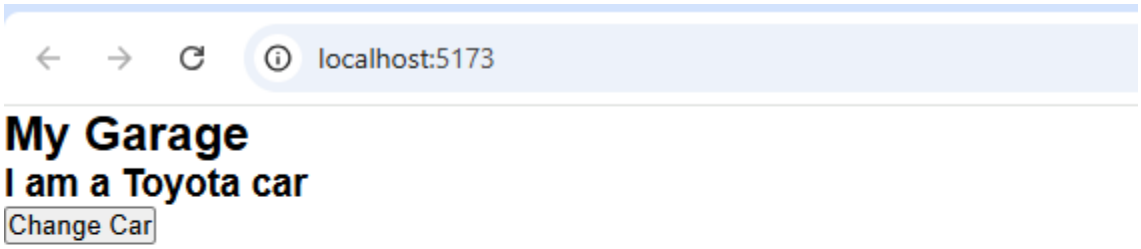
// Garage component (uses state + passes props)
function Garage() {
  const [carName, setCarName] = useState("Toyota");
```

```
  return (
    <div>
      <h2>My Garage</h2>

      {/* Passing state as props */}
      <Car brand={carName} />

      {/* Change state */}
      <button onClick={() => setCarName("BMW")}>Change Car</button>
    </div>
  );
}

export default Garage;
```



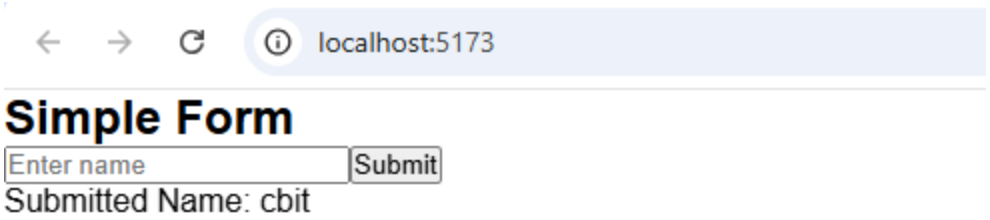
Program 3: Implement a React Controlled Form with Input State Handling

```
import React, { useState } from "react";

function ReactForm() {
  const [name, setName] = useState('');
  const [submittedName, setSubmittedName] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmittedName(name); // Save input to display
    setName(''); // Clear input
  };
 return (
    <div>
      <h2>Simple Form</h2>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Enter name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
        <button type="submit">Submit</button>
      </form>
      {submittedName && <p>Submitted Name: {submittedName}</p>}
    </div>
  );
}
export default ReactForm;
```

Output:



Program4: Implementing Basic Routing Example in React Using React Router

In one file

```
import React from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

import {First,Home,About} from './RouterD'

function RouterComp() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/home">Home</Link> | <Link to="/about">About</Link>
      </nav>

      <Routes>
        <Route path="/" element={<First />} />
        <Route path="/home" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}   export default RouterComp;
```

In second file

```
import React from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";


function First()
{
    return <h2>Welcome to the First Page</h2>;
}
function Home() {
  return <h2>Welcome to the Home Page</h2>;
}

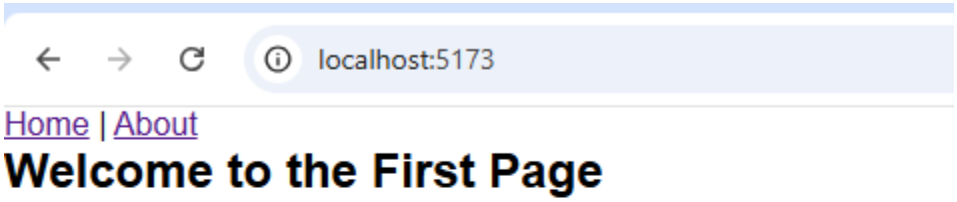
function About() {
  return <h2>This is the About Page</h2>;
}

export {First, Home, About};
```

Running steps

npm install react-router-dom

Npm run dev





Home | About

Welcome to the Home Page

Program 5: Implement Pagination Concept on the data available in table

```
import React from "react";
import ReactTable from "react-table-6";
import "react-table-6/react-table.css";

function TablePagination() {
  const data = [
    { id: 1, name: "John", age: 25 },
    { id: 2, name: "Jane", age: 30 },
    { id: 3, name: "Sam", age: 22 },
    { id: 4, name: "Chris", age: 28 },
    { id: 5, name: "Alex", age: 35 },
    { id: 6, name: "Emma", age: 27 },
    { id: 7, name: "Olivia", age: 24 },
    { id: 8, name: "Liam", age: 29 },
  ];

  const columns = [
    {
      Header: "ID",
      accessor: "id",
    },
    {
      Header: "Name",
      accessor: "name",
    },
    {
      Header: "Age",
      accessor: "age",
    },
  ];

  return (
```

```
    <div>
      <h2>React Table v6 Pagination</h2>

      <ReactTable
        data={data}
        columns={columns}
        defaultPageSize={3}    // number of rows per page
        pageSizeOptions={[3, 5, 10]} // dropdown options
        showPagination={true} // enable pagination
      />
    </div>
  );
}

export default TablePagination;
```

Program 6: Create a React component that checks the strength of a password and displays the result to the user. The component will take user input and use a set of rules to determine the strength of the password using with Regular Expression and without regular expressions.

```jsx
import React, { useState } from 'react';

function PasswordStrengthChecker() {
  const [password, setPassword] = useState('');
  const [strength, setStrength] = useState('');

  const checkStrength = (pwd) => {
    let score = 0;

    if (pwd.length >= 8) score++;
    if (/[A-Z]/.test(pwd)) score++;
    if (/[a-z]/.test(pwd)) score++;
    if (/[0-9]/.test(pwd)) score++;
    if (/[^A-Za-z0-9]/.test(pwd)) score++;

    switch (score) {
      case 5:
        return 'Very Strong 💪';
      case 4:
        return 'Strong ✅';
      case 3:
        return 'Medium ⚠️';
      case 2:
        return 'Weak ❌';
      default:
        return 'Very Weak ❌';
    }
  };

  const handleChange = (e) => {
    const pwd = e.target.value;
    setPassword(pwd);
    setStrength(checkStrength(pwd));
  };

  return (
    <div style={{ maxWidth: '400px', margin: '20px auto', fontFamily:
'Arial' }}>
      <h2>Password Strength Checker</h2>
```

```
      <input
        type="password"
        value={password}
        onChange={handleChange}
        placeholder="Enter your password"
        style={{ width: '100%', padding: '10px', marginBottom: '10px' }}
      />

      {password && (
        <p>
          Strength: <strong>{strength}</strong>
        </p>
      )}
    </div>
  );
}

export default PasswordStrengthChecker;
```

Do With Regular Expressions also

WeeK-8

1) Design the authorized end points using JWT (JSON Web Token)

Aim

To create a **secure REST API** using **JSON Web Tokens (JWT)** that allows users to log in, receive a token, and access protected data based on their identity. The API ensures that only authorized users can access their specific posts.

Description

The code implements a **basic authentication and authorization system** using JWT in a Node.js environment with Express. It performs the following key tasks:

1. **Login Endpoint (`/login`)**
   - Accepts a username from the client.
   - Creates a JWT token containing the username as the payload.
   - Returns the token to the client.
   - The token acts as a **digital key** to access protected resources.
2. **JWT Middleware (`authenticateToken`)**
   - Checks for a token in the **Authorization header** of incoming requests.
   - Verifies the token using a **secret key** from `.env`.
   - If valid, allows the request to proceed; otherwise, denies access (401/403).
   - Attaches the decoded user info to the request for route handling.
3. **Protected Endpoint (`/posts`)**
   - Only accessible with a valid JWT.
   - Filters and returns posts based on the logged-in user's name.
   - Ensures users see only their own data.
4. **Server Setup**
   - Express server listens on port 3000.
   - Uses `express.json()` for parsing incoming JSON requests.
   - Demonstrates **stateless authentication**, where user session info is stored in the JWT, not on the server.

Execution Procedure:

Step -1:
mkdir jwt-project
cd jwt-project

Step-2

npm init -y

Step-3

npm install express jsonwebtoken dotenv

Step -4
Create Project Files

Create server.js – main code file
Create .env – store secret key:

.env file content

ACCESS_TOKEN=mysecretkey123

Step-5 observe that Folder structure should look like this:

jwt-project/
├── server.js
├── .env
├── package.json
└── node_modules/

Step-6

Write the JWT code into `server.js`.
Ensure you have:

- **`/login` endpoint for generating token**
- **`authenticateToken` middleware**
- **Protected `/posts` endpoint**

Code:

```
const express = require('express') // Import Express framework
const app = express()              // Create an Express app instance
const jwt = require('jsonwebtoken')// Import JSON Web Token library for
authentication

require('dotenv').config()         // Load environment variables from .env
file

app.use(express.json())            // Middleware to parse JSON bodies from
incoming requests

// Sample posts data - each post has a name and a title
const posts = [
  {
    name: "CBIT",
    title: "Welcome to CBIT"
  },
  {
    name: "MGIT",
    title: "Welcome to MGIT"
  }
]

// Middleware to authenticate JWT tokens
const authenticateToken = (req, res, next) => {
  // Get the Authorization header (expected format: "Bearer <token>")
  const authHeader = req.headers['authorization']
  // Extract the token part from the header
```

```
  const token = authHeader && authHeader.split(' ')[1]

  // If no token found, respond with 401 Unauthorized
  if (!token) return res.sendStatus(401)

  // Verify the token with the secret key from environment variables
  jwt.verify(token, process.env.ACCESS_TOKEN, (err, user) => {
    if (err) {
      // If token verification fails, respond with 403 Forbidden
      return res.sendStatus(403)
    }
    // Attach decoded user info to request object for use in next handlers
    req.user = user
    next() // Continue to next middleware or route handler
  })
}

// POST /login route to authenticate user and issue a JWT token
app.post('/login', (req, res) => {
  const username = req.body.username // Extract username from request body
  const user = { name: username }    // Create a user payload with the
username
  // Sign a JWT token with user payload and secret key
  const accessToken = jwt.sign(user, process.env.ACCESS_TOKEN)
  // Send the token back in JSON response
  res.json({ token: accessToken })
})

// Apply authentication middleware to all routes below this line
app.use(authenticateToken)

// GET /posts route - returns posts filtered by authenticated user's name
app.get('/posts', (req, res) => {
  console.log(req.user.name) // Log the authenticated username
  // Filter posts to only those matching the logged-in user and send JSON
response
  res.json(posts.filter(post => post.name === req.user.name))
})

// Start the Express server on port 3000
```

```
app.listen(3000, () => console.log("Server running on port 3000"))
```

Step 7: Start the Server

1. In the terminal, run:

```
Bash
node server.js
```

2. You should see:

```
Server running on port 3000
```

Step 8: Open Postman to Test APIs

A. Login (Generate JWT Token)

1. Open Postman → Click New Request → POST
2. URL:

http://localhost:3000/login

3. Go to Body → raw → JSON, enter:

{
"username": "CBIT"
}

4. Click Send
5. ✅ Response will be like:

{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}

This is your JWT token. Copy it.

B. Access Protected Posts

1. New GET request → URL:

   http://localhost:3000/posts

2. Go to Authorization → Type: Bearer Token, paste your token.
   OR add header manually:

   Authorization: Bearer YOUR_TOKEN

3. Click Send
4. ✅ Response will show only posts matching the logged-in username:

```
[
 {
  "name": "CBIT",
  "title": "Welcome to CBIT"
 }
]
```

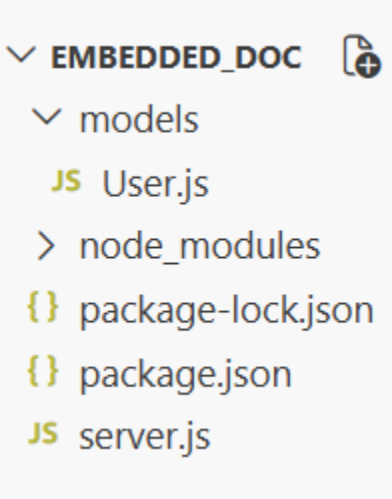
C. Test Error Cases

1. No Token → Remove Authorization header → Send → Response: `401 Unauthorized`
2. Invalid Token → Modify token → Send → Response: `403 Forbidden`
3. Different User → Login as `MGIT` → Send → Only MGIT posts appear

Week-9

Program: Storing a Document with Embedded Subdocuments in MongoDB using Mongoose

Folder Structure



User.js

```
const mongoose = require("mongoose");
const { Schema, model } = mongoose;

// Subdocument Schema (Address)
const addressSchema = new Schema({
  street: String,
  city: String,
  country: String,
});

// Main Schema (User)
const userSchema = new Schema({
  name: String,
```

```
  email: String,
  addresses: [addressSchema]    // 👈 Embedded subdocuments
});

// Model
const User = model("User1", userSchema);

module.exports = User;
```

Server.js

```
const mongoose = require("mongoose");
const User = require("./models/User");

async function run() {
  try {
    // Connect MongoDB
    await mongoose.connect("mongodb://127.0.0.1:27017/testdb1");
    console.log("Connected to MongoDB");

    // Create User with embedded addresses
    const user = new User({
      name: "Swathi",
      email: "Swathi@example.com",
      addresses: [
        { street: "123 Main St", city: "New York", country: "USA" },
        { street: "456 Elm St", city: "Boston", country: "USA" }
      ]
    });

    // Save document
    await user.save();
    console.log("User saved successfully!");

    // Fetch all users
    const users = await User.find();
    console.log("All Users:", users);
```

```
  } catch (err) {
    console.log("Error:", err.message);
  } finally {
    // Disconnect MongoDB
    await mongoose.disconnect();
    console.log("Disconnected from MongoDB");
  }
}

// Run function
run();
```

Execution steps

Embedded_doc> npm init -y
Embedded_doc> npm install mongoose
Embedded_doc>mongod
Mongodb server will run

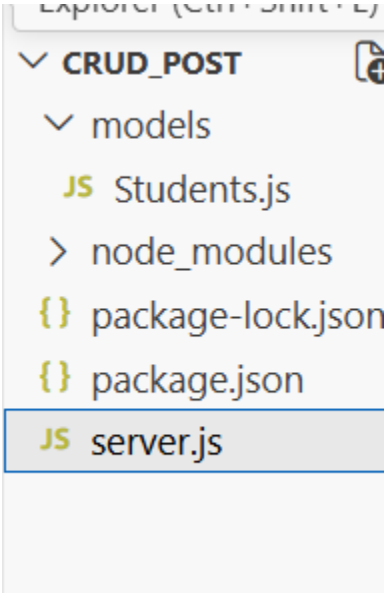
Open one more terminal
Embedded_doc> node server.js

```
Connected to MongoDB
User saved successfully!
All Users: [
  {
    _id: new ObjectId('69de4e828b529040ba4bcea0'),
    name: 'Swathi',
    email: 'Swathi@example.com',
    addresses: [ [Object], [Object] ],
    __v: 0
  },
  {
    _id: new ObjectId('69e06544edb21ac95045e50d'),
    name: 'Swathi',
    email: 'Swathi@example.com',
    addresses: [ [Object], [Object] ],
    __v: 0
```

```
  }
]
Disconnected from MongoDB
```

Program:2– Develop a backend application with REST API to perform CRUD operations on student data. (Use Postman Tool)

Folder Structure



Students.js

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema({
  rollNo: { type: String, required: true, unique: true },
  name: String,
  className: String
});

module.exports = mongoose.model("Student", studentSchema);
```

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(express.json());
app.use(cors());

// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/studentDB123")
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));

const Student = require("./models/Students");


// ➤ CREATE Student
app.post("/students", async (req, res) => {
  try {
    const student = new Student(req.body);
    await student.save();
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});


// ➤ READ All Students
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});


// ➤ READ Single Student
app.get("/students/:rollNo", async (req, res) => {
  const student = await Student.findOne({ rollNo: req.params.rollNo
});
```

```
  if (!student) return res.status(404).json({ message: "Not found"
});
  res.json(student);
});


// ➤ UPDATE Student
app.put("/students/:rollNo", async (req, res) => {
  const student = await Student.findOneAndUpdate(
    { rollNo: req.params.rollNo },
    req.body,
    { new: true }
  );
  res.json(student);
});


// ➤ DELETE Student
app.delete("/students/:rollNo", async (req, res) => {
  await Student.findOneAndDelete({ rollNo: req.params.rollNo });
  res.json({ message: "Student deleted" });
});


// Server Start
app.listen(5050, () => {
  console.log("Server running on port 5000");
});
```

Execution steps

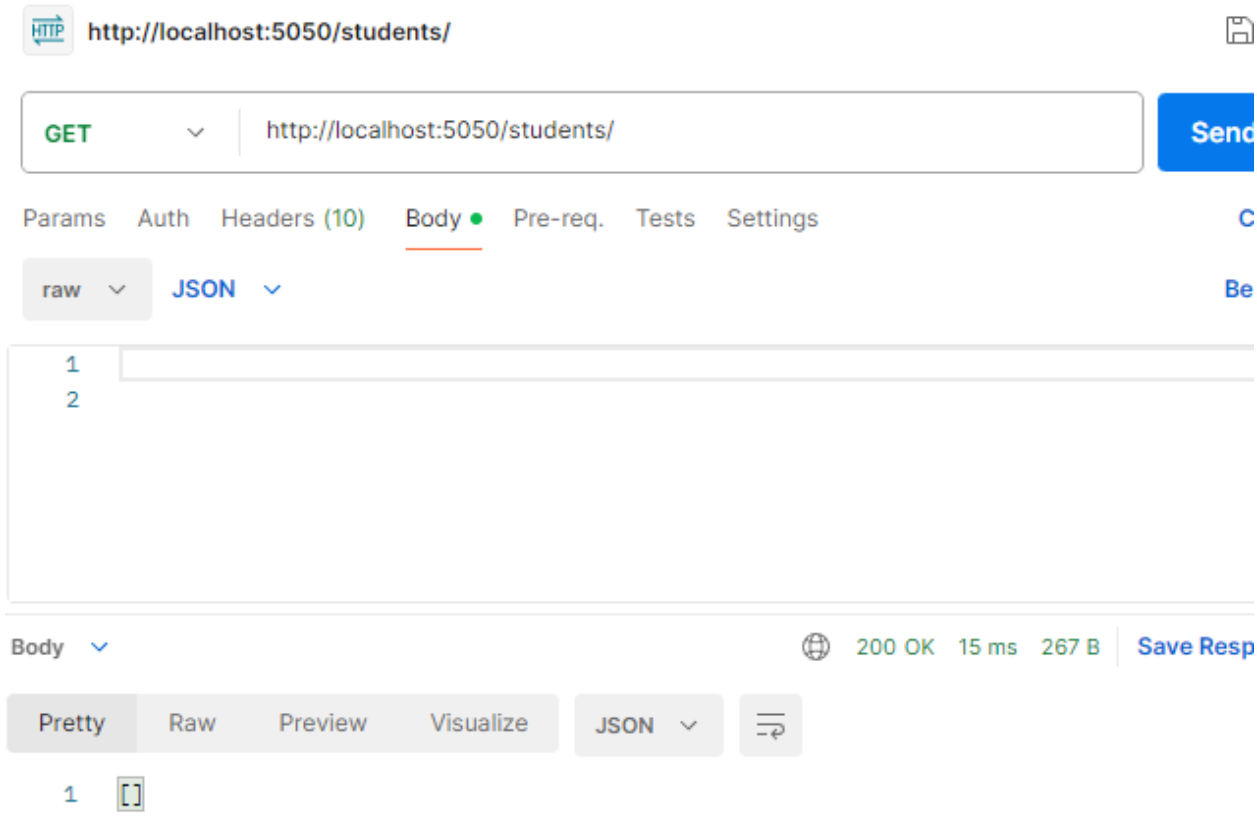
Step-1

CRUD_POST> npm init -y

CRUD_POST> npm install cors express mongoose

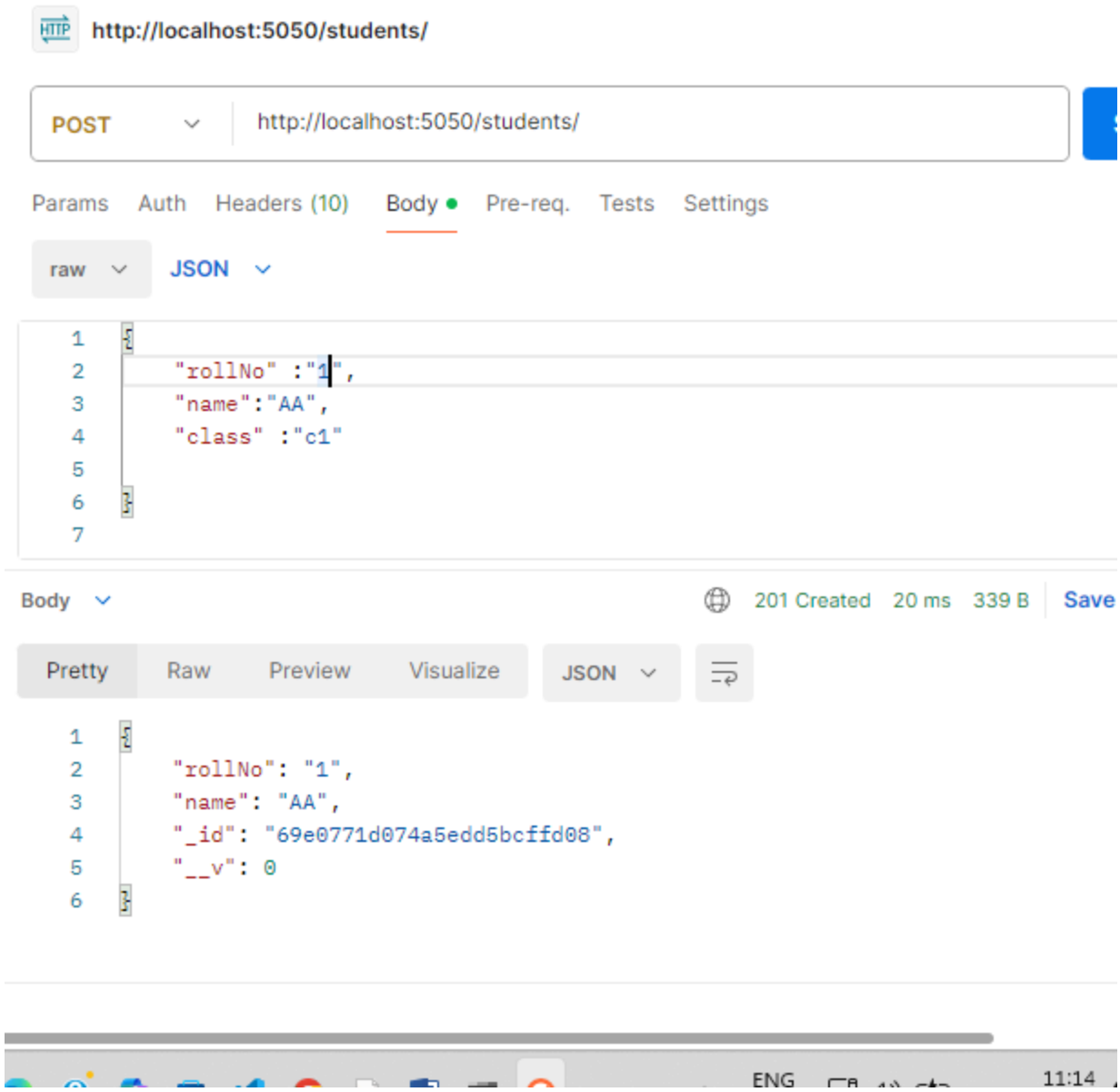
Step-2

Open postman tool and check all the CRUD methods.

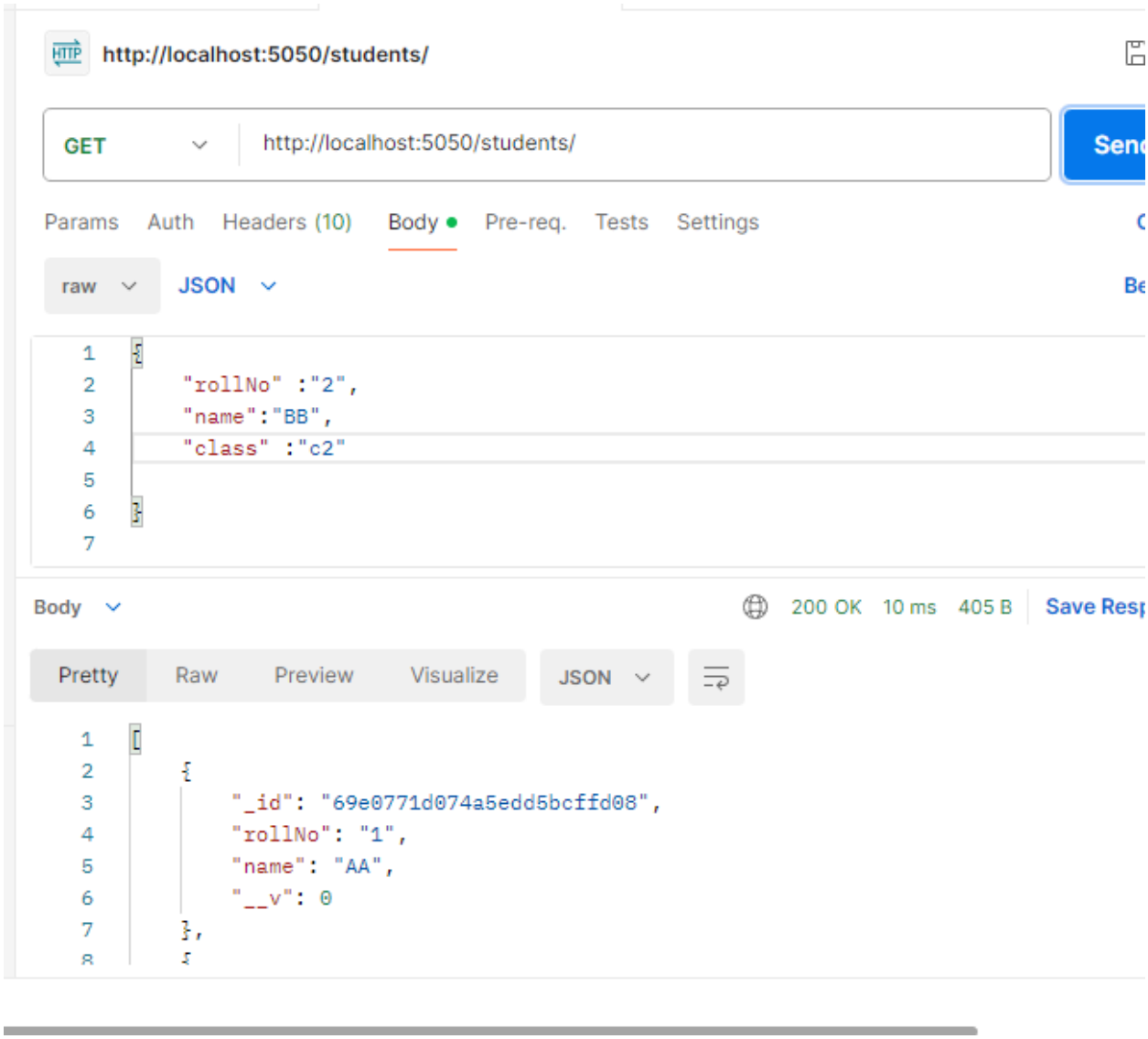


No data available

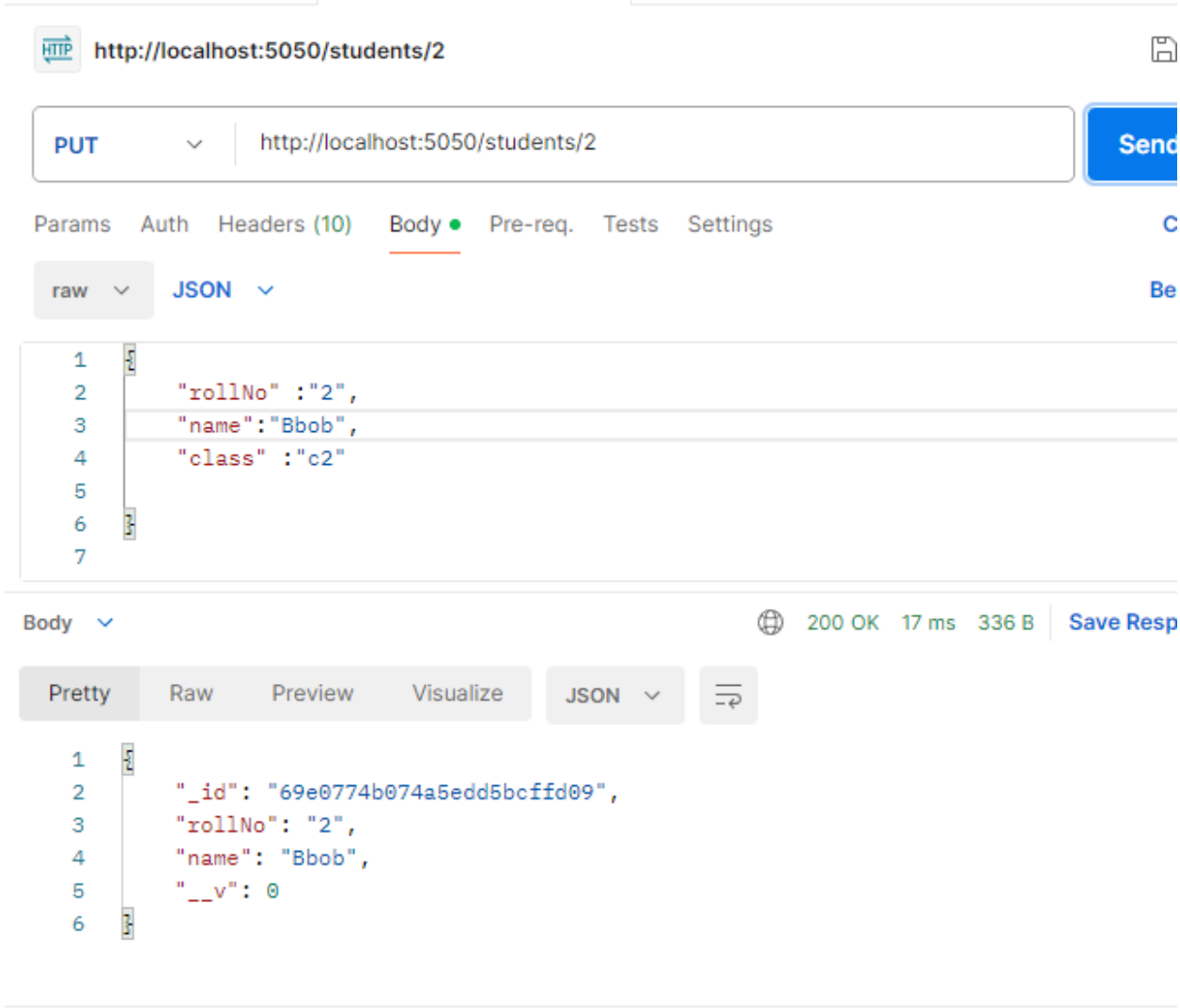
Inserting data



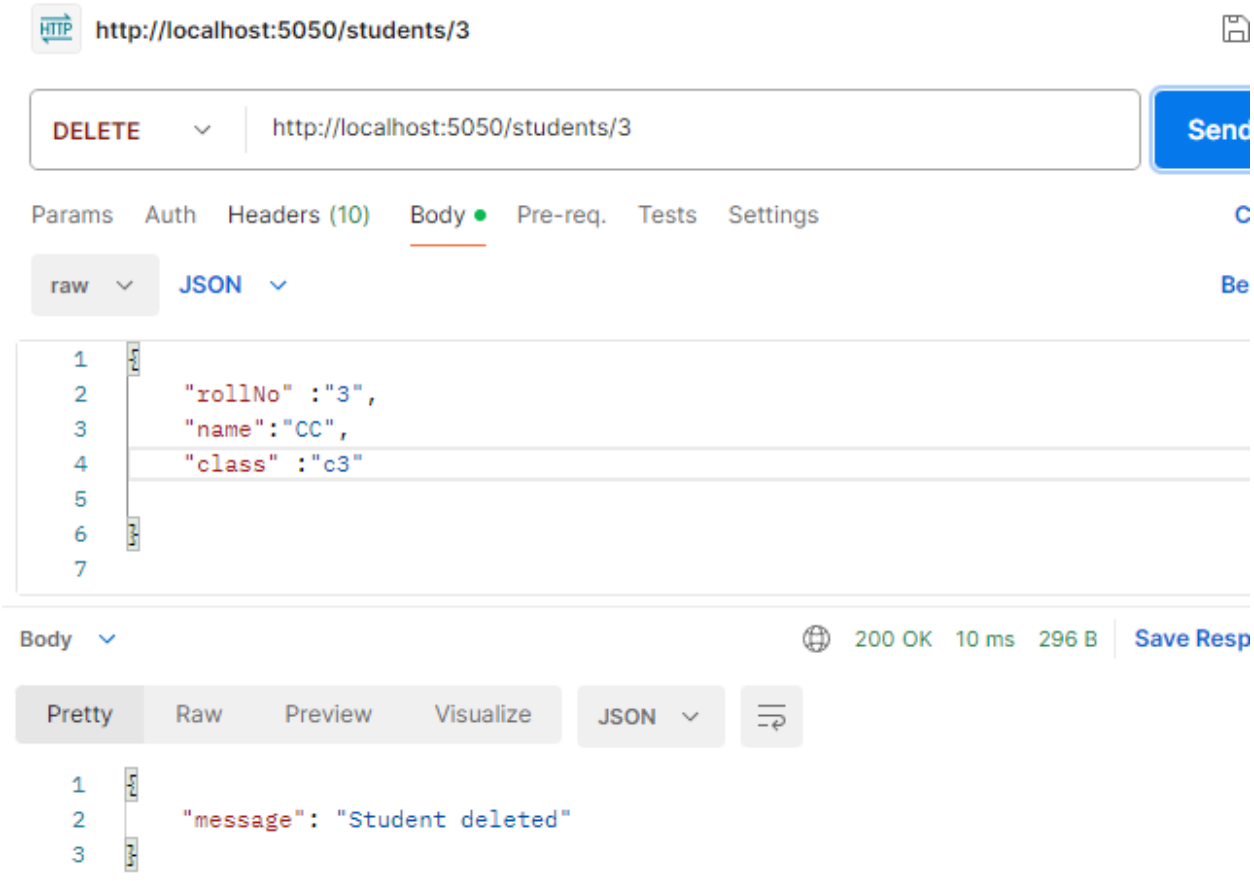
Retrieving the available data



Update the RollNo:02 data



Delete the Roll No:3 document


http://localhost:5050/students/3
DELETE
http://localhost:5050/students/3
Send
Params
Auth
Headers (10)
Body
Pre-req.
Tests
Settings
raw
JSON
{
"rollNo" :"3",
"name":"CC",
"class" :"c3"
}
Body
200 OK
10 ms
296 B
Save Resp
Pretty
Raw
Preview
Visualize
JSON
{
"message": "Student deleted"
}

Week-10

Design replica set of student database and insert records in primary node and display the records in secondary nodes.

Aim:

To configure and demonstrate a MongoDB Replica Set, enabling data replication across multiple MongoDB instances and verifying data availability on secondary nodes.

Description:
In this task, three MongoDB instances are started on different ports (27020, 27021, 27022) with replication enabled using the `--replSet` option. These instances act as members of a replica set named m101.

The replica set is initialized from the primary node (Terminal 1) using a configuration object that defines all members. Once initialized, MongoDB automatically elects a primary node and designates the others as secondary nodes.

A sample document is inserted into a collection (`College`) on the primary node. The replication mechanism ensures that this data is copied to the secondary nodes. By connecting to the secondary instances and enabling read operations (`rs.secondaryOk()`), the replicated data can be verified.

Finally, the primary node is shut down to observe the replica set's fault tolerance. The system automatically handles failover by electing a new primary from the remaining nodes, ensuring high availability and data redundancy.

Code:

TERMINAL1

```
start mongod --replSet m101 -logpath \data\rs1\1.log --dbpath \data\rs1
--port 27020
```

TERMINAL2

```
start mongod --replSet m101 -logpath \data\rs2\2.log --dbpath \data\rs2
--port 27021
```

TERMINAL3

```
start mongod --replSet m101 -logpath \data\rs3\3.log --dbpath \data\rs3
--port 27022
```

TERMINAL1

```
mongosh --port 27020

config = { _id: "m101", members:[
          { _id : 0, host : "localhost:27020"},
          { _id : 1, host : "localhost:27021"},
          { _id : 2, host : "localhost:27022"} ]
         };


rs.initiate(config);
rs.status();

db.College.insert( { CName: "CBIT", Dept: "IT" } )
db.College.find();
```

TERMINAL2

```
mongosh --port 27021
rs.secondaryOk()
db.College.find();
```

TERMINAL3

```
mongosh --port 27022
rs.secondaryOk()
db.College.find();
```

TERMINAL1

```
use admin;
db.shutdownServer();



open a TERMINAL2 & TERMINAL3

rs.status();
```

Displaying MongoDB Replica Updated.txt.